TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN
KHOA KỸ THUẬT MÁY TÍNH

ĐẶC TẢ ĐỀ TÀI VÀ KẾ HOẠCH 13 TUẦN

Đồ án 1 – Học kỳ 1, năm học 2026–2027 (Mốc M0)

1. Thông tin nhóm và đề tài

Mã nhóm	N17
Loại đồ án	Đồ án 1
GVHD	Nguyễn Duy Xuân Bách
Thành viên	Huỳnh Thanh Sang – 23521341 Nguyễn Hoàng Hải Minh – 24521066
Tên đề tài	Research on the deployment and evaluation of µEDP runtime infrastructure for edge AI applications in the acoustic predictive maintenance problem of bearings. (Nghiên cứu triển khai và đánh giá hạ tầng điều phối thực thi µEDP cho ứng dụng AI tại biên trong bài toán bảo trì tiên đoán âm học vòng bi)
Hướng cụ thể	Dùng µEDP (framework hướng sự kiện, mô hình Active Object) làm runtime điều phối pipeline suy luận AI ở vi điều khiển cho bài toán bảo trì tiên đoán bằng dấu vết âm học trên đối tượng vòng bi: thu âm I2S → trích MFCC → suy luận mô hình int8 → cảnh báo. Đo latency, jitter, footprint và độ chính xác, so sánh với các kết quả nghiên cứu hiện hành.
Mã nguồn	Repo µEDP (runtime), repo CE201 (demo đề tài), repo PdM-AF (phần cứng thu âm và dữ liệu tham chiếu)

2. Bài toán và mục tiêu

2.1. Bài toán

Hỏng đột ngột ở thiết bị quay như vòng bi, động cơ gây dừng máy và mất an toàn lao động. Bảo trì tiên đoán bằng âm thanh cho phép giám sát không tiếp xúc và phát hiện bất thường từ sớm. Để chạy được ngay trên thiết bị biên, hệ thống phải xử lý luồng âm thanh tần số cao (dự kiến 32kHz) trong điều kiện RAM hạn chế, đồng thời vẫn phải làm các tác vụ thời gian thực khác.

Các RTOS phổ biến như FreeRTOS hay Zephyr dùng nhiều luồng, mỗi luồng một ngăn xếp riêng và có tính chất chặn (blocking). Mô hình hướng sự kiện (QP/C, AK-EDP) tránh blocking bằng cách dùng Active Object và truyền tin nhắn. µEDP là framework hướng sự kiện do nhóm tự phát triển theo mô hình này, nhưng chưa có nghiên cứu nào đánh giá việc triển khai một hạ tầng điều phối với suy luận AI. Đề tài dùng bài toán PdM bằng âm thanh làm bài toán thử để trả lời 2 câu hỏi: việc sử dụng một runtime framework có khả thi cho

suy luận AI tại biên hay không? µEDP có thể là một lựa chọn mới giữa các hạ tầng truyền thống như FreeRTOS hay Zephyr trong việc xây dựng hệ thống nhúng đa dụng không?

2.2. Mục tiêu

- Xây dựng thiết bị biên chạy µEDP, thu âm máy quay bằng microphone, trích MFCC và phân loại trạng thái bình thường / bất thường theo thời gian thực ngay trên vi điều khiển.
- Tổ chức pipeline thành 4 Active Object: TASK_ACQ (thu âm), TASK_FEAT (trích MFCC), TASK_INFER (suy luận), TASK_DISP (cảnh báo/hiển thị).
- Đo latency, jitter, footprint RAM/Flash và độ chính xác; so sánh với baseline không có nền tảng điều phối hoặc sử dụng các nền tảng điều phối như kernel Linux và FreeRTOS.

2.3. Đối tượng sử dụng kết quả

Nhóm nghiên cứu hệ thống nhúng và Edge AI cần một bảng số liệu tham chiếu về việc đưa ứng dụng TinyML vào một runtime điều phối; người làm bảo trì thiết bị cần một thiết bị giám sát âm thanh chạy tại chỗ.

3. Input – Output

3.1. Input

Dữ liệu vào	Nguồn	Định dạng / tần số	Ghi chú
Âm thanh máy quay (thời gian thực)	Microphone I2S trên thiết bị	PCM theo khung ngắn; tần số lấy mẫu dự kiến 32 kHz	Thu bằng DMA, kích hoạt qua ngắt
Âm thanh huấn luyện (công khai)	UOEMD-VAFCVS (42kHz), UORED-VAFCLS (42kHz) và GUET Multi-Condition Acoustic Ball Bearing (32,768kHz)	UOEMD-VAFCVS, UORED-VAFCLS: file .mat/.csv đa kênh (gồm kênh micro, rung, tải, tốc độ), 42kHz, 10s/file (420.000 mẫu); GUET: file .bkc (định dạng B&K), chỉ kênh âm thanh, 32.768kHz, đoạn liên tục dài (chia file 10 phút)	Huấn luyện nền tảng offline
Âm thanh tự thu	Vòng bi thí nghiệm theo quy trình của repository PdM-AF	Cùng định dạng thu của thiết bị	Bổ sung/tinh chỉnh cho đúng điều kiện phần cứng
Cấu hình µEDP	Bảng task, kích thước message pool	Kconfig / µE-LS	Chỉnh khi thực nghiệm

3.2. Output

Sản phẩm	Nội dung	Cách thể hiện
Firmware trên µEDP	4 Active Object chạy trên STM32H723 driver I2S trong tầng PAL	Mã nguồn trên Git
Mô hình AI	MLP hoặc CNN 1D nhỏ trên đặc trưng MFCC, lượng tử hoá int8; thư viện CMSIS-NN cho STM32H723	Accuracy, precision, recall trên tập test giữ lại
Kết quả suy luận	Trạng thái bình thường / bất thường, cảnh báo tại chỗ	Hiển thị/ghi log trên thiết bị
Bảng benchmark	Latency, jitter từng task và toàn pipeline; RAM/Flash; µEDP so với baseline kết quả nghiên cứu đã có	Bảng số liệu đo trên phần cứng thật
Demo	Kịch bản chạy end-to-end với trạng thái bình thường và bất thường	Video 1–3 phút, tài liệu build/nạp

4. Phạm vi

4.1. Làm trong học kỳ này

- Khảo sát runtime/RTOS, framework hướng sự kiện, TinyML cho PdM và acoustic fingerprint; lập bảng so sánh, xác định khoảng trống.
- Khôi phục và bổ sung tầng PAL cho STM32H723.
- Pipeline AI: huấn luyện MLP/CNN 1D trên MFCC, lượng tử hoá PTQ int8, triển khai bằng C/C++ (không dùng Python trên thiết bị).
- Cài MFCC bằng số học cố định trên MCU (CMSIS-DSP) và đối chiếu với bản float32 dùng khi huấn luyện.
- Đo benchmark và so sánh với baseline FreeRTOS hoặc bare-metal.

[Đề xuất] Vì thời gian chỉ có 13 tuần, ưu tiên chạy đầy đủ end-to-end trên STM32H723 kiến trúc đơn nhân trước. ESP32-S3 với kiến trúc đa nhân từng nằm trong danh thiết bị sẽ dự kiến dành cho đồ án 2; phần còn lại chuyển sang khóa luận.

4.2. Không làm

- Không nghiên cứu kiến trúc Deep Learning mới; không dùng mô hình lớn (ResNet, Transformer).
- Không phát triển thêm hệ điều hành hướng sự kiện (µE-OS) hoặc mở rộng tính năng lõi µEDP ngoài phần cần cho bài toán AI.
- Không thử nghiệm ở môi trường công nghiệp thật; chỉ dùng lỗi cơ khí giả lập trong phòng thí nghiệm.
- Không truyền dữ liệu lên cloud; cảnh báo và hiển thị tại chỗ.
- Không dùng QAT trừ khi PTQ làm độ chính xác giảm quá ngưỡng chấp nhận.

5. Sơ đồ khối hệ thống

Luồng xử lý trên thiết bị (bản nháp, sẽ vẽ lại bằng hình khi nộp):

Microphone I2S → DMA/ngắt → TASK_ACQ → TASK_FEAT (MFCC) → TASK_INFER (mô hình int8) → TASK_DISP (cảnh báo)

Task	Nguồn kích hoạt	Vai trò
TASK_ACQ	Ngắt DMA I2S, qua cơ chế Safe ISR Injection	Thu khung âm thanh thô, là điểm vào của hệ thống
TASK_FEAT	Message từ TASK_ACQ	Trích MFCC bằng số học cố định
TASK_INFER	Message từ TASK_FEAT	Chạy mô hình đã lượng tử hóa trên vector đặc trưng
TASK_DISP	Message từ TASK_INFER	Xuất cảnh báo / hiển thị kết quả

Phía PC có một pipeline độc lập chạy offline: trích MFCC bằng Python (librosa hoặc python_speech_features), huấn luyện bằng TensorFlow/Keras, lượng tử hoá và chuyển đổi thành artifact .tflite hoặc bộ trọng số cho CMSIS-NN. Pipeline này chỉ tồn tại ở giai đoạn phát triển, không đưa vào firmware.

Các tác vụ giao tiếp bằng message pool tĩnh. Do bộ lập lịch không chiếm quyền và chạy theo run-to-completion, thời gian suy luận một lần của TASK_INFER cộng trực tiếp vào độ trễ chờ của các task khác. Vì vậy cần giới hạn thời gian suy luận theo ngân sách, và nếu vượt thì chia nhỏ bằng máy trạng thái lai (HSMC) của µEDP.

6. Tiêu chí đạt và cách đo

Ghi chú: các ngưỡng trong ngoặc vuông là đề xuất, sẽ chốt với thầy và sau khi có số đo đầu tiên.

Chỉ tiêu	Giá trị cần đạt	Cách đo
Độ chính xác mô hình	Báo cáo accuracy, precision, recall trên tập test giữ lại; F1 ≥ [90]%; nêu mức tụt từ float32 sang int8	Tách tập train/test từ trước; đánh giá trên PC và trên board
Sai lệch MFCC (train–serve skew)	Sai lệch giữa MFCC fixed-point trên MCU và MFCC float32 ≤ [ngưỡng chốt sau khi đo thử]	Đưa cùng một khung âm thanh qua cả hai cách cài đặt, so sánh vector đặc trưng
Latency, jitter	Có số liệu từng task và toàn pipeline khi task AI chạy xen các task thời gian thực khác	Bật/tắt GPIO + logic analyzer hoặc timer phần cứng; chạy liên tục ≥ [30] phút, ghi trung bình, lớn nhất, độ lệch chuẩn
Đảm bảo thời gian thực	Không mất khung âm thanh, không tràn message pool trong lúc chạy	Đếm số khung bị bỏ và số lần cấp phát message thất bại
Footprint	Báo cáo RAM/Flash của mô hình và của toàn firmware	Đọc từ file map/size sau khi build
So sánh baseline	µEDP so với FreeRTOS hoặc bare-metal, Linux, cùng bài toán, cùng phần cứng	Cùng mô hình, cùng dữ liệu, cùng cách đo
Demo	Nhận ra trạng thái bất thường trong kịch bản thử; số lần thử chốt với thầy	Chạy thử với máy / vòng bi bình thường và có lỗi giả lập, ghi lại kết quả

7. Thiết bị và phần mềm

Loại	Danh mục	Tình trạng
Board STM32	Kit LXBH723ZGP1V2 (STM32H723ZGT6)	Đã sẵn có
Module I2S	PCM1808 96kHz	Đã sẵn có
Module âm thanh	MAX9812	Đã sẵn có
Module khuếch đại tín hiệu	TL072IDT	Đã sẵn có
Đối tượng đo	Vòng bi thí nghiệm, có cách tạo lỗi giả lập	Chưa có tương ứng với các tình trạng cần phân loại
Dụng cụ đo	Logic analyzer hoặc chân GPIO + oscilloscope	Chưa có
Phần mềm	Python, TensorFlow/Keras, TensorFlow Lite Converter, TensorFlow Lite Micro, CMSIS-NN, CMSIS-DSP, librosa, µE-LS (YAML, Jinja2, Kconfig), FreeRTOS (baseline), Git	Miễn phí / có sẵn

8. Kế hoạch 13 tuần và phân công

8.1. Phân công thành viên

Thành viên	Phần việc chính
Huỳnh Thanh Sang	Lõi runtime µEDP, tích hợp inference engine vào scheduler, thiết kế và đo benchmark (latency, jitter, footprint) so với baseline, bổ sung API PAL cho STM32H723.
Nguyễn Hoàng Hải Minh	Pipeline AI (thu thập dữ liệu → huấn luyện → chuyển đổi/lượng tử hoá → đánh giá), ứng dụng demo end-to-end.
Cả nhóm	Khảo sát tài liệu (chia theo nhóm chủ đề ở Mục 10), viết báo cáo, thống nhất giao diện giữa runtime và pipeline AI (định dạng message, kích thước khung, vector đặc trưng).

8.2. Bảng kế hoạch

Ghi chú: S = Sang, M = Minh.

Tuần	Thời gian	Mốc nộp	Công việc	Phụ trách	Đầu ra
1	28/09 – 04/10	M0 (05/10)	Chốt tên đề tài và hướng với thầy; viết đặc tả và kế hoạch; xác định phần repo cũ tái dùng được; chốt release µEDP	S, M	Bản đặc tả M0
2	05/10 – 11/10		Khảo sát theo nhóm chủ đề (Mục 10); rà lại tầng PAL hiện có của board	S: nhóm B, C M: nhóm A, D	Ghi chú khảo sát từng bài
3	12/10 – 18/10	M1 (19/10)	Hoàn thiện bảng so sánh, rút khoảng trống, chọn hướng; chọn bộ dữ liệu; chốt tần số lấy mẫu, kích thước khung	S, M	Báo cáo khảo sát (≥ 8 tài liệu, có trích dẫn)
4	19/10 – 25/10		Khôi phục port PAL STM32H723, thêm driver I2S. Dựng harness đo (GPIO), chốt cấu hình task table và message pool. Tiền xử lý dữ liệu, trích MFCC bằng Python.	M S M	PAL + I2S chạy được Harness đo Đặc trưng offline
5	26/10 – 01/11		Baseline FreeRTOS thu âm + MFCC trên STM32H723 Huấn luyện mô hình float32, lượng tử hoá PTQ int8	S M	Baseline chạy được Mô hình int8 + mức tụt chính xác
6	02/11 – 08/11	M2 (09/11)	TASK_ACQ + TASK_FEAT chạy trên STM32H723 thật (thu âm I2S, MFCC fixed-point); đối chiếu với Python; quay video	S, M	Module cốt lõi chạy thật; video; link Git
7	09/11 – 15/11		Tích hợp TASK_INFER với TFLite Micro vào scheduler Tự thu dữ liệu vòng bi, tinh chỉnh mô hình	S M	TASK_INFER chạy được Dữ liệu tự thu đợt 1
8	16/11 – 22/11		TASK_DISP, ghép đủ 4 task; đo sơ bộ latency/jitter	S M	Pipeline 4 task chạy liền mạch
9	23/11 – 29/11	M3 (30/11)	Tích hợp end-to-end trên STM32H723; lập danh sách lỗi còn tồn tại; quay video	S, M	Prototype chạy luồng chính; danh sách lỗi

Tuần	Thời gian	Mốc nộp	Công việc	Phụ trách	Đầu ra
10	30/11 – 06/12		Đo benchmark µEDP so với baseline trên STM32H723 Đánh giá mô hình float32 và int8 trên tập test và trên board	S M	Bảng benchmark Bảng độ chính xác
11	07/12 – 13/12	M4 (14/12)	Phân tích sai lệch so với mục tiêu; chạy pipeline trên ESP32-S3 nếu kịp; viết bản nháp báo cáo	S, M	Bảng đo đầy đủ; bản nháp báo cáo
12	14/12 – 20/12		Hoàn thiện báo cáo, slide; đóng gói mã nguồn và hướng dẫn chạy; tập demo	S, M	Báo cáo, slide, repo đầy đủ
13	21/12 – 27/12	M5 (28/12)	Chỉnh sửa lần cuối, quay video demo, nộp báo cáo cuối	S, M	Bộ hồ sơ M5

9. Rủi ro và phương án dự phòng

Rủi ro	Ảnh hưởng	Phương án dự phòng
Phạm vi lớn (2 board, AI, benchmark)	Không kịp hoàn thành	Ưu tiên STM32H723 end-to-end; ESP32-S3 làm phần PAL, phần còn lại để cho khóa luận
Port PAL chưa có trên nhánh chính, thiếu API và driver I2S	Trễ mốc M2	Làm PAL và I2S ngay tuần 4; dùng dữ liệu phát lại để phát triển task khác song song
MFCC fixed-point trên MCU lệch so với float32 khi huấn luyện	Mô hình tốt khi train nhưng kém khi chạy thật	Đối chiếu đặc trưng sớm (tuần 6); huấn luyện lại bằng đặc trưng đã tính theo cách cài đặt trên MCU nếu cần
Suy luận chiếm quyền làm tăng jitter của task khác	Không đạt yêu cầu thời gian thực	Giới hạn ngân sách thời gian suy luận; chia nhỏ bằng HSMC; giảm kích thước mô hình
RAM không đủ cho bộ đệm âm thanh, MFCC và mô hình	Không nạp được mô hình	Giảm tần số lấy mẫu, số hệ số MFCC hoặc kích thước mô hình; tận dụng SD card module của kit STM32H723
Dữ liệu công khai không phải vòng bi và khác điều kiện thu (domain shift)	Độ chính xác thấp trên dữ liệu thật	Thu thêm dữ liệu tại phòng thí nghiệm; huấn luyện nền từ dữ liệu công khai rồi tinh chỉnh
Release µEDP thay đổi trong lúc làm	Lệch số liệu, phải làm lại	Chốt một release ổn định ở tuần 1

10. Danh sách nguồn dự kiến khảo sát (riêng Đồ án 1)

Danh sách 16 tài liệu từ literature review của nhóm, chia thành 5 nhóm: A (TinyML/Edge AI cho PdM), B (acoustic sensing và fingerprinting), C (kiến trúc IoT/nhúng chi phí thấp), D (phương pháp luận PdM), E (khảo sát tổng quan). Các mục [6]–[16] cần bổ sung nơi đăng và DOI trước khi nộp M1.

STT	Nhóm	Nguồn	Dùng để làm gì
[1]	A	S. Gupta, S. N. Shivhare, Embedded TinyML for Predictive Maintenance: Vibration Analysis on ESP32 with Real-Time Fault Detection in Industrial Equipment, Int. J. Comput. Model. Appl., 2025	CNN 1D nhỏ, int8 trên ESP32
[2]	E	O. Vermesan et al. (Eds.), Industrial Artificial Intelligence Technologies and Applications, River Publishers, 2022	Bối cảnh Industrial AI
[3]	A	Z. Chen, Y. Gao, J. Liang, LOPdM: A Low-Power On-Device Predictive Maintenance System Based on Self-Powered Sensing and TinyML, IEEE Trans. Instrum. Meas., 2023	Đánh đổi độ chính xác và năng lượng
[4]	C	P. Kolok et al., Low-Cost IoT-Based Predictive Maintenance Using Vibration, Sensors, 2025	Baseline phần cứng ESP32 chi phí thấp
[5]	A	P. Vitolo et al., Low-Power Detection and Classification for In-Sensor Predictive Maintenance Based on Vibration Monitoring, IEEE Sensors J., 2022	Giới hạn phần cứng tại cảm biến
[6]	B	A. Das, N. Borisov, M. Caesar, Do You Hear What I Hear?: Fingerprinting Smart Devices Through Embedded Acoustic Components, 2014	Khái niệm acoustic fingerprint
[7]	B	Y. Wang, Audio Data Acquisition System Design Based on ARM and DSP, 2015	Thiết kế thu âm
[8]	D	S.-Y. Chuang et al., Predictive Maintenance with Sensor Data Analytics on a Raspberry Pi-Based Experimental Platform, 2019	Pipeline thu thập – phân tích – cảnh báo
[9]	E	P. Nunes, J. Santos, E. Rocha, Challenges in Predictive Maintenance – A Review, 2022	Thách thức PdM, giới hạn đề tài
[10]	C	I. T. Franco, R. M. de Figueiredo, Predictive Maintenance: An Embedded System Approach, 2022	Ràng buộc hệ nhúng
[11]	E	V. Kumar, K. Paul, Device Fingerprinting for Cyber-Physical Systems: A Survey, 2023	Định vị fingerprinting
[12]	A	E. Omol et al., Anomaly Detection in IoT Sensor Data Using Machine Learning Techniques for Predictive Maintenance in Smart Grids, 2024	Phát hiện bất thường trên dữ liệu cảm biến

STT	Nhóm	Nguồn	Dùng để làm gì
[13]	C	E. García-Ortega, Application of Embedded Systems in Industrial IoT for Predictive Maintenance, 2024	Ràng buộc triển khai IIoT
[14]	E	L. Cummins et al., Explainable Predictive Maintenance: A Survey of Current Methods, Challenges and Opportunities, 2024	Hướng mở rộng
[15]	B	J. Nagy, I. Lakatos, Acoustic Fingerprint in Vehicle Manufacturing as a Basis for Future Applications, 2025	Ứng dụng acoustic fingerprint trong sản xuất
[16]	D	A. Arregi, A. Barrutia, I. Bediaga, End-to-End Methodology for Predictive Maintenance Based on Fingerprint Routines and Anomaly Detection for Machine Tool Rotary Components, 2025	Quy trình fingerprint → anomaly detection

Ngoài ra, phần khảo sát runtime và triển khai AI trên MCU sẽ dùng tài liệu của QP/C (M. Samek), FreeRTOS, Zephyr Project, TensorFlow Lite for Microcontrollers và CMSIS-NN; bộ dữ liệu UOEMD-VAFCVS, UORED-VAFCLS và GUET Multi-Condition Acoustic Ball Bearing cùng các bài báo mô tả chúng.

11. Báo cáo tiến độ

Việc đã hoàn thành:

1. Tổng hợp toàn bộ tài liệu bài báo nghiên cứu khoa học chung chủ đề và thực hiện literature review hoàn chỉnh
2. Thống nhất tên đề tài và hướng đi của nghiên cứu tiến tới khóa luận
3. Thống nhất tần số thu thập mẫu của hệ thống
4. Chốt kit phần cứng và lộ trình sử dụng theo khóa luận
5. Tổng hợp và chốt các bộ dataset sử dụng cho đề tài
6. Bổ sung mô phỏng thiết kế mạch khuếch đại tín hiệu nâng nền áp trên LTSpice
7. Thống nhất danh sách linh kiện sơ bộ để xây dựng testbench của hệ thống
8. Chốt phiên bản v1.1.6-bf1 cho uEDP.
9. Hoàn thiện sơ bộ tài liệu pipeline training model cho hệ thống
10. Bổ sung CMSIS-NN cho repository chính và chỉnh sửa CMake khớp với setup của repository

Việc đang thực hiện / vướng mắc:

1. Hiện tại cần phải chỉnh sửa lại mạch khuếch đại tín hiệu và nền điện áp tín hiệu của schematics cũ theo mô phỏng mới
2. Xác định thông số motor sử dụng làm testbench để kiểm tra phù hợp với hệ thống

Cần hỗ trợ: đề xuất motor sử dụng và các tham số cho testbench trên các linh kiện như trục vít me, giá đỡ vòng bi, kích thước vòng bi

Dự kiến tiếp theo:
1. Bổ sung API wrapper cho CMSIS-NN để tương thích với styling code của uEDP.
2. Triển khai schematics và PCB mới để trình bày và thực hiện gia công mạch
3. Bổ sung đầy đủ tài liệu phụ trợ datasheet cho các module như PCM1808, TL072IDT, MAX9812

Repository: https://github.com/1811htsang/CE201.git